

Consumer Behavior Analysis in the Beauty Industry

Dain Lee, Rameena Quddus, Sloka Jagannathan

November 2025

1 Abstract

This study uses two large datasets of Ulta beauty product metadata and customer reviews to examine consumer behavior, product performance, and brand impression in the cosmetics business. Three clustering algorithms (K-Means, Hierarchical Clustering, and K-Medoids) were used to group products based on pricing, engagement, and rating patterns after a thorough data preprocessing and exploratory analysis. Four significant product groups that highlighted different behavior and popularity groups throughout the market were consistently found by all three methods. The textual sentiment of the reviews was quantified and aligned with projected ratings using a BERT-based sentiment model (nlptown/bert-base-multilingual-uncased-sentiment). The model successfully distinguished between positive and negative emotions on a scale with an accuracy of 74.72%, successfully distinguishing between positive and negative emotional tone. Sentiment patterns aligned with qualitative trends found in the reviews: positive reviews emphasized qualities such as smooth application and long-lasting wear, while negative reviews commonly referenced dryness, patchiness, shade mismatch, and issues with product formulation.

The combined studies identify the top and bottom performers, show distinct variations in consumer participation in product segments, and offer useful information for future dashboard or recommendation systems, brand strategy, and product evaluation.

2 Introduction

The rise of e-commerce over the last several decades has fundamentally transformed how consumers interact with brands. Earlier eras of beauty retail were defined by in-store sampling, counter demonstrations, and mass-media advertising. Today, however, online marketplaces and digital communities play a central role in shaping expectations for product quality, authenticity, and usability. Modern consumers rarely make decisions based solely on brand advertising; instead, they rely on detailed product descriptions, recommendation systems, short-form videos, and, most importantly, user-generated content such as reviews and ratings through social media and websites.

Online marketing has evolved from static banner advertising to experiential content like reviews, tutorial content, and short-form video on platforms such as TikTok [2]. Influencers also play an important role in shaping modern consumer behavior and affect consumer purchase intentions through their perceived attributes, such as credibility, attractiveness, and authenticity, which enhance brand trust and consumer willingness to buy [5]. Data-driven personalization is another strategy with a major contribution. Recommendation systems of platforms like Netflix and Amazon utilize past purchases, searches, clicks, and forsaken carts to suggest personalized recommendations and thereby improve satisfaction, trust, and retention [2].

Customer satisfaction is one of the critical drivers of online marketplace success. Kotler and Keller (2016) define satisfaction as a person's feeling of pleasure or disappointment resulting from comparing perceived performance with expectations [4]. Five key factors influencing satisfaction in online marketplaces include customer service, fulfillment/reliability, ease of use, products/services, and security/privacy. The cosmetics industry has become a highly data rich environment in which consumer preferences, product performance,

and brand reputation are shaped by large volumes of online information. Because sensory experiences like texture, makeup finish, and shade cannot be evaluated physically in online environments, reviews serve well to inform customers of product performance. Furthermore, Chen et al. (2022) demonstrated that consumers allocate more visual attention to negative reviews than positive ones, suggesting that negative online feedback carries disproportionate weight in shaping purchase intentions [3]. This makes them an essential component for companies to establish trust with customers, particularly in the beauty domain where personal preference and skin compatibility are highly individualized.

Retailers such as Ulta Beauty have extensive collections of makeup products that include detailed product attributes along with hundreds of thousands of customer reviews. As online shopping continues to grow, it becomes important to analyze these data in order to identify trends, understand consumer needs, and guide the development of more effective product and marketing strategies. Data driven analytical methods offer a powerful way to explore these patterns and anticipate customer demands [1]. Clustering techniques can group products based on shared characteristics such as price, rating, shade range, and engagement metrics. These methods can reveal natural product segments and help identify which groups of products attract the most attention or show the strongest signs of consumer satisfaction [6].

The sentiment from text contained in customer reviews also provides deeper insight into how users describe their experiences. Simple rating averages may mask important nuances, such as a product receiving an average 4-star rating but still having consistent complaints about shade inclusivity or packaging, specific product issues that traditional metrics could overlook. Sentiment analysis allows researchers to move beyond rating numbers and uncover the emotional tone that consumers express when discussing their makeup routines, product performance, and overall satisfaction. Transformer-based models are particularly well suited for this task because they capture linguistic context and subtle emotional cues, offering a more precise understanding of consumer attitudes.

The importance of analyzing this type of consumer feedback has grown alongside broader trends in digital marketing. Platforms increasingly integrate machine learning techniques to tailor product discovery, prioritize relevant items, and filter out products that exhibit persistently negative sentiment. As a result, extracting insights from large review datasets is no longer merely descriptive—it is foundational to competitive strategy, brand positioning, and customer retention within the beauty marketplace.

In this study we analyze two large Ulta datasets that contain detailed product information and more than three hundred thousand written reviews. After completing extensive preprocessing and exploratory data analysis we examined patterns of missingness, distribution shapes, correlations, and text structure in order to understand the overall behavior of the data. Three clustering approaches were then applied including K Means, Hierarchical Agglomerative Clustering, and K Medoids. All three methods consistently revealed four meaningful product clusters. These clusters represented distinct levels of consumer engagement, pricing patterns, and rating behavior which indicates that the marketplace contains natural groupings of products that differ in popularity and performance. A BERT-based (Bidirectional Encoder Representations from Transformers) sentiment model was also used to classify the emotional tone of the review text. The classifier assigned a sentiment score on a one to five scale and achieved an accuracy of more than seventy four percent. This analysis highlighted recurring themes in both positive and negative reviews.

Together, these analyses provide a comprehensive view of consumer behavior in the cosmetics market. By combining clustering, textual sentiment, and exploratory analysis this study offers insight into what consumers value most in makeup products and where the most common sources of dissatisfaction arise. These results also highlight opportunities for improvement in formulation, shade variety, texture consistency, and product communication which can help strengthen brand reputation and guide future product development.

3 Method

3.1 Data Processing

To ensure consistency across features, all numeric variables were preprocessed prior to analysis. Missing values were imputed using column medians to preserve the underlying distribution of each variable. Columns

containing only a single unique value were removed because they did not contribute meaningful information to downstream modeling.

Several engagement-related variables (e.g., review counts and star-level rating counts) displayed strong right-skew. These features were transformed using the `log1p` function to reduce the influence of extreme values and improve comparability across variables. After all transformations, numeric features were standardized using the `StandardScaler` method, placing attributes such as price, number of shades, average rating, and review volume on comparable scales so that each feature contributed proportionally during clustering.

3.2 Exploratory Data Analysis

Exploratory data analysis (EDA) was performed to build an initial understanding of the structure and characteristics of both the products and reviews datasets. The analysis began with identifying patterns of missingness to inform imputation and feature-cleaning decisions.

Subsequent EDA examined rating distributions across product categories, temporal trends in review activity, and engagement indicators such as review length, helpfulness votes, and star-level rating counts. Text-focused EDA, including word-frequency patterns in positive and negative reviews, was also conducted to understand how customers described their experiences.

These exploratory steps clarified the structure and quality of the datasets, guided preprocessing decisions, and helped determine which variables were appropriate for clustering and sentiment modeling.

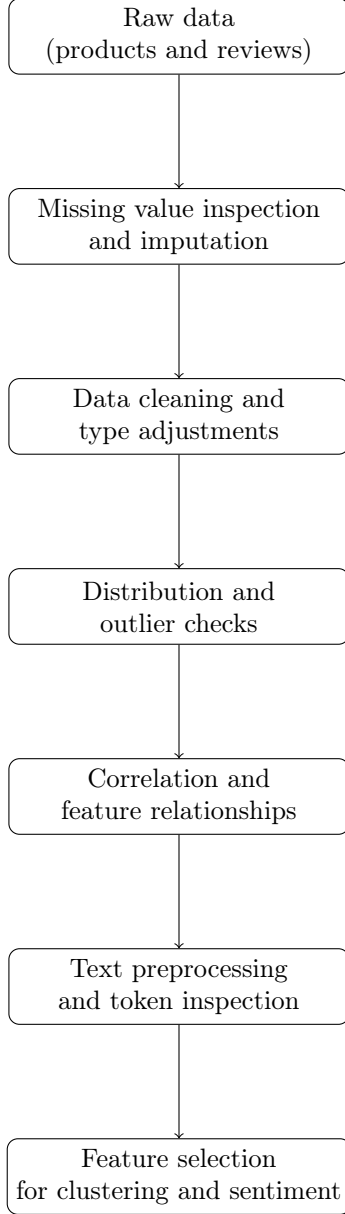


Figure 1: Exploratory data analysis workflow used before clustering and sentiment modeling.

3.3 Statistical and Machine Learning Methods

3.3.1 Clustering

Three clustering algorithms were applied to the standardized numeric dataset to identify meaningful product segments. K Means clustering was used first, and the optimal number of clusters was evaluated using the Elbow Method and the Silhouette metric. Hierarchical Agglomerative Clustering was then implemented with Ward linkage and Euclidean distance, and dendrograms were generated to visualize how product groups merged as dissimilarity increased. A third method, K Medoids implemented through the pyclustering library, was included to enhance robustness by selecting actual observations as cluster centers. For all three algorithms, model quality was assessed through Elbow curves and Silhouette scores, and both two dimensional

and three dimensional PCA projections were produced to visualize the resulting cluster separations.

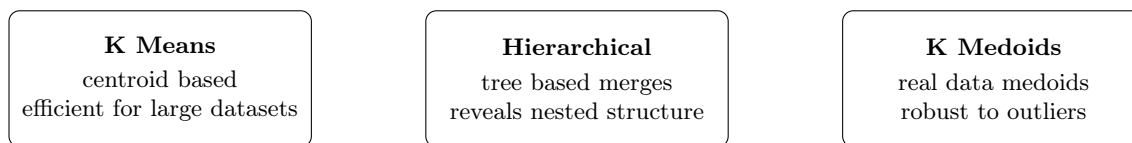


Figure 2: Conceptual comparison of the three clustering algorithms used in this study.

3.3.2 Sentiment Modeling

Review text underwent several cleaning steps, including conversion to lowercase, removal of punctuation and digits, elimination of stopwords, and filtering of domain specific redundant terms. The cleaned text was then processed by a BERT based sentiment classifier from the nlptown model family, which generated predicted sentiment scores on a five point scale. Predictions of three or higher were categorized as positive sentiment, while predictions below three were treated as negative. Model performance was evaluated using overall accuracy and a confusion matrix that compared predicted sentiment with known positive and negative review subsets.

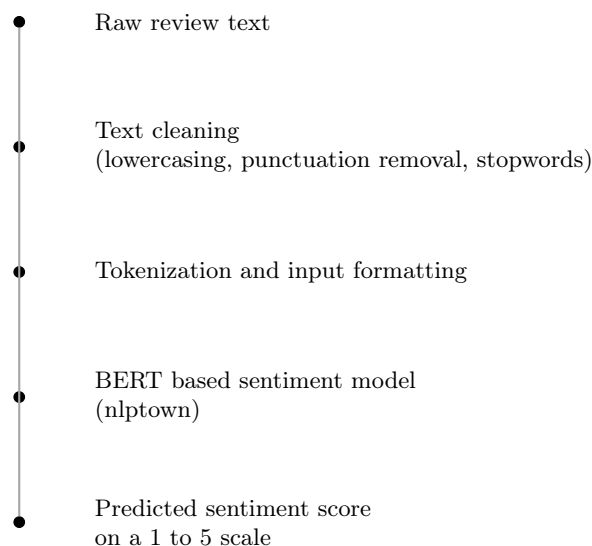


Figure 3: Timeline of preprocessing and modeling steps used for BERT based sentiment classification.

4 Results

4.1 Correlation Analysis

To understand how product attributes relate to one another before applying clustering models, we examined a pairwise scatterplot matrix of key numeric variables (Figure 4). Total reviews, native reviews, and syndicated reviews are examples of engagement-related metrics that show clear positive correlations with one another, creating a cohesive collection of popularity and user activity indicators. The diagonal histograms further show that review volumes and helpfulness-related features are highly right-skewed, supporting the decision to apply log transformations during preprocessing.

In contrast, price and number of shades show weaker and more diffuse relationships with engagement variables, suggesting that popularity does not strongly depend on price or product variety. Rating also

remains only mildly associated with other features, reinforcing that quality perceptions and product visibility behave as largely independent dimensions. These patterns confirm the presence of multiple underlying feature clusters and justify the use of standardized numeric variables for downstream clustering.

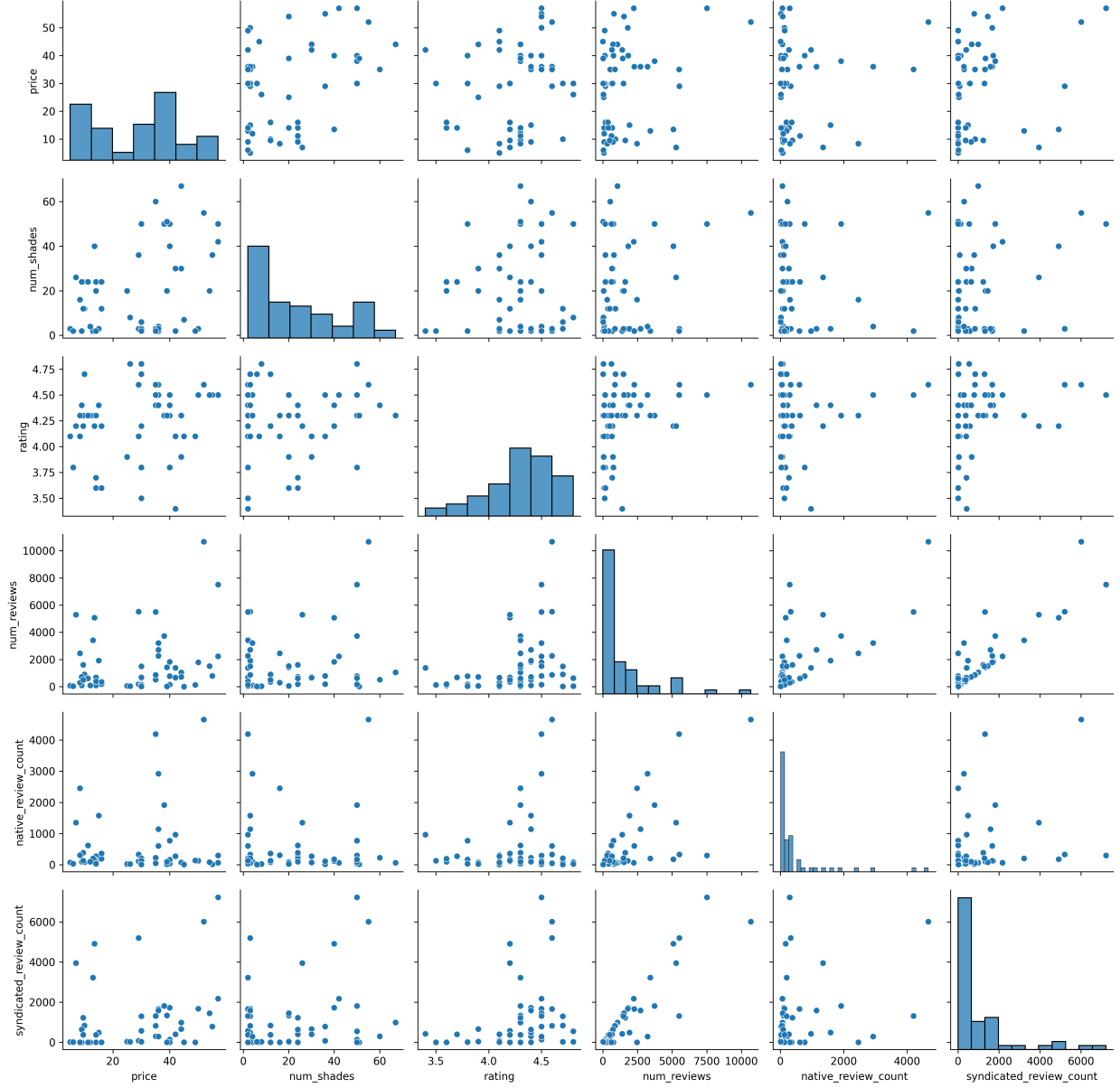


Figure 4: Pairplot showing relationships among price, shades, rating, and engagement metrics, with diagonal distributions highlighting the dataset's structure before clustering.

4.2 Clustering Results

4.2.1 Clustering - Exploratory Data Anal

To build an initial understanding of the product dataset, we conducted an exploratory analysis focusing on distributional patterns, feature relationships, and potential underlying structure. Several numeric variables—including price, number of shades, average rating, and multiple engagement metrics—were examined first to characterize their raw behavior. Engagement-related variables such as total review count, native review count, and syndicated review count displayed strong right-skewed distributions, indicating that only a small subset of products attracts very large volumes of consumer interaction. As shown in Figure 5, applying a log transformation substantially reduced this skew, compressing extreme values and yielding a more balanced distribution that is more appropriate for distance-based modeling.

To evaluate whether the dataset contained meaningful multivariate structure, we performed an exploratory Principal Component Analysis (PCA) on the standardized numeric features. The two-dimensional PCA projection in Figure 6a shows clear dispersion rather than a single dense cluster, suggesting that products differ along multiple latent dimensions. This observation provides early evidence that clustering will be able to detect distinct segments rather than producing trivial or overlapping groupings.

Finally, to explore linear relationships between variables, we calculated absolute pairwise correlations and extracted the strongest feature associations. As shown in Figure 6b, engagement variables formed the most pronounced correlated pairs, highlighting a shared behavioral dimension related to overall product visibility and consumer interest across platforms. In contrast, product attributes such as price, number of shades, and rating exhibited comparatively weak correlations with engagement, suggesting that the drivers of popularity extend beyond inherent product characteristics.

Together, these findings reveal a dataset with clear structural properties: engagement metrics that move closely together, product-related attributes that vary independently, and improved distributional balance achieved through preprocessing. These insights informed the selection of features used in clustering and provided the empirical foundation for subsequent modeling analyses.

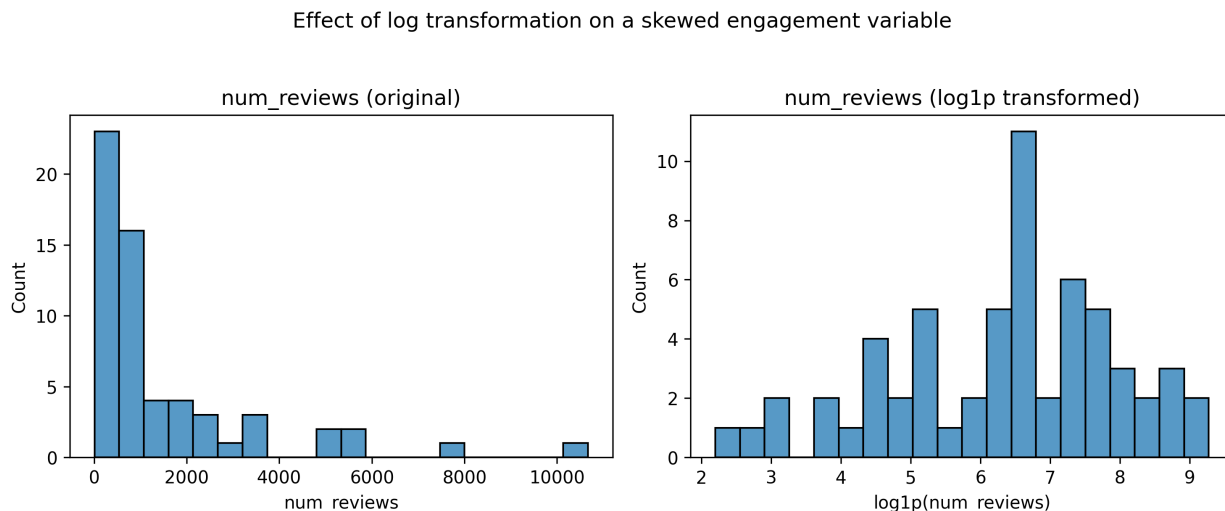
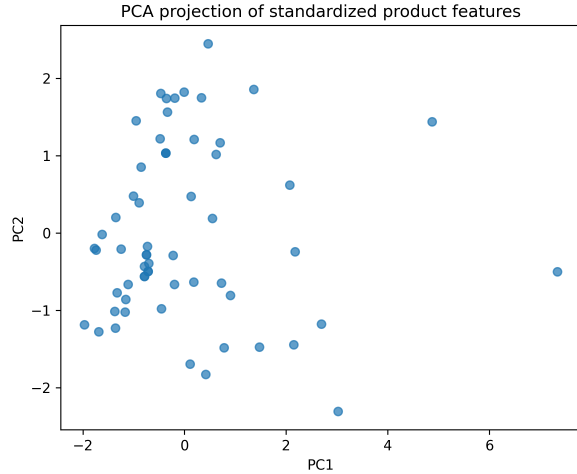
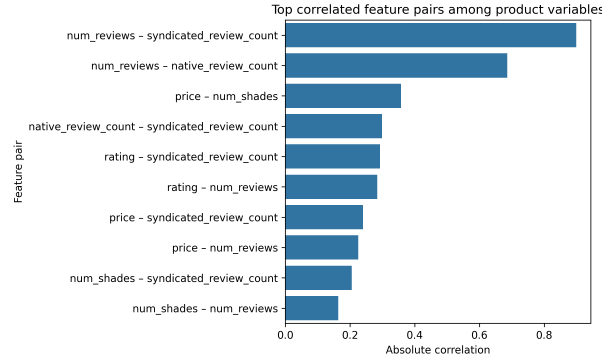


Figure 5: Effect of log transformation on a skewed engagement variable.



(a) PCA projection of standardized product features.



(b) Top correlated feature pairs among product variables.

Figure 6: Exploratory PCA and correlation patterns.

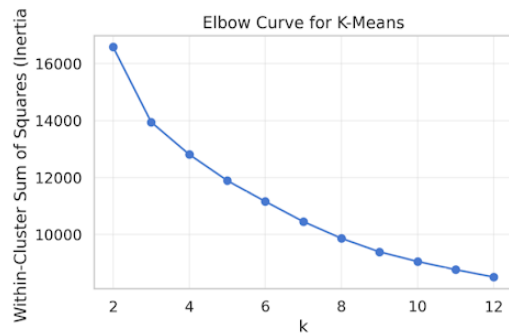
4.2.2 K Means

K-Means clustering was first applied to the standardized numeric product features to explore whether the dataset contained coherent groupings based on engagement behavior, pricing, and product characteristics. To determine an appropriate value for k , we evaluated model fit using both the Elbow and Silhouette methods.

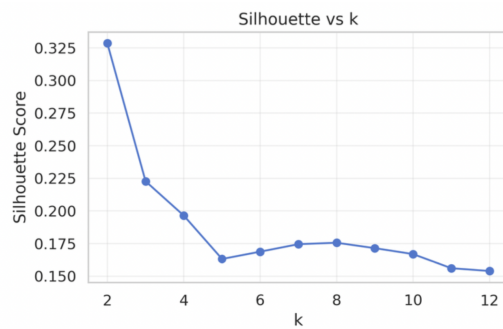
The Elbow curve (Figure 7a) shows a steep decline in within-cluster inertia from $k = 2$ to $k = 4$, after which the rate of improvement slows noticeably. This “bend” indicates that additional clusters beyond four contribute diminishing explanatory benefit. The Silhouette scores across different values of k (Figure 7b) further support this choice: although $k = 2$ yields the highest score, performance plateaus beginning at $k = 4$, suggesting that four clusters provide a more balanced trade-off between cluster separation and granularity.

Based on these diagnostics, we selected $k = 4$ as the final configuration. The resulting clusters were visualized using PCA-based projections. The 2D PCA plot (Figure 8a) shows clear spatial separation between the four clusters, indicating that the underlying standardized features contain well-defined latent structure. The silhouette distribution plot for $k = 4$ (Figure 8b) also shows predominantly positive silhouette coefficients, confirming that the clusters are cohesive and reasonably well-distinguished from one another. The 3D PCA visualization (Figure 8c) yields a similar pattern, demonstrating that separation persists even when more variance is captured in higher-dimensional space.

Together, these results indicate that K-Means identifies four stable, interpretable product groups, capturing distinct combinations of engagement intensity, review volume, price levels, and rating patterns. These clusters form the basis for later comparisons with sentiment outcomes and brand-level trends.

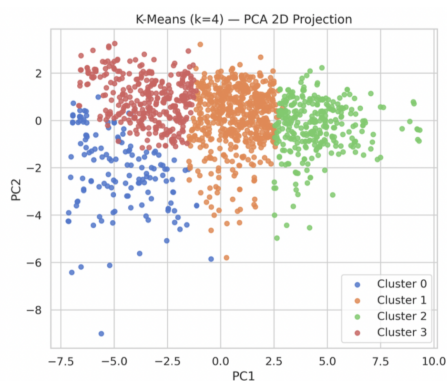


(a) Elbow curve

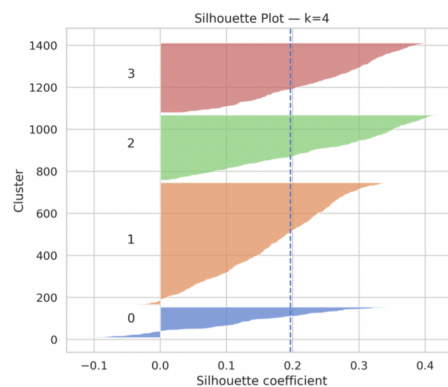


(b) Silhouette score vs. k

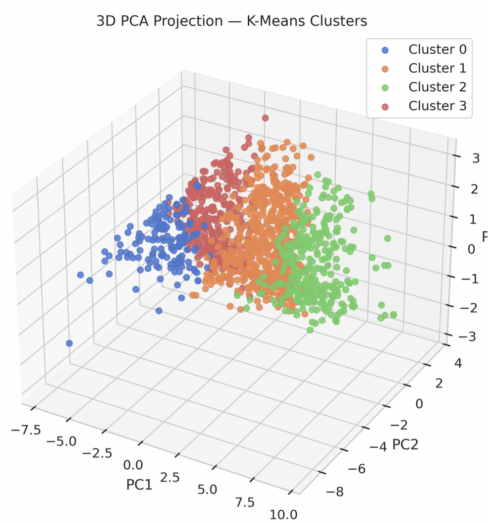
Figure 7: Model selection diagnostics for choosing k in K-Means.



(a) 2D PCA projection



(b) Silhouette plot for $k = 4$



(c) 3D PCA projection

Figure 8: Cluster visualizations for the final $k = 4$ K-Means solution.

4.2.3 Hierarchical Clustering

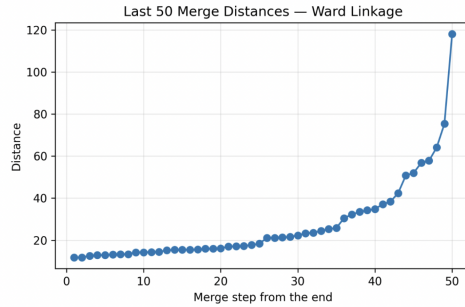
Hierarchical agglomerative clustering using Ward linkage was applied to the standardized product features to evaluate whether a tree-based merging structure would reveal patterns similar to the K-Means results. This method is particularly useful for examining how clusters successively combine at increasing distances, providing a complementary perspective on the dataset’s latent organization.

To assess the appropriate number of clusters, we examined the merge distances across the final stages of the hierarchy. The last 50 merge distances (Figure 9a) show a pronounced increase near the final merges, indicating that combining the remaining clusters requires substantially larger jumps in within-cluster variance. This sharp rise suggests that a natural break exists around four clusters.

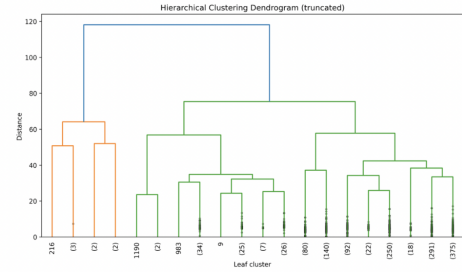
A truncated dendrogram (Figure 9b) further illustrates this structure. The tree shows clear branching patterns, with several subgroups merging at relatively low distances while a small number of major branches remain separate until much higher distances. This supports the idea that the dataset contains several dense, cohesive subpopulations.

Cluster quality was then evaluated using silhouette analysis. The silhouette-by- k plot (Figure 9c) shows that silhouette scores decline sharply after $k = 2$ but remain relatively stable around $k = 4$, mirroring the pattern observed in the K-Means diagnostics. The silhouette distribution for $k = 4$ (Figure 9d) shows that clusters 2 and 3 exhibit strong cohesion, while clusters 0 and 1 contain more borderline points—an expected outcome because hierarchical methods cannot reassign points once merged.

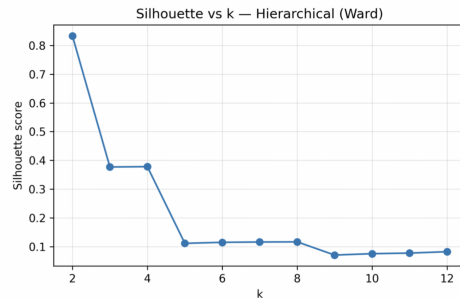
Taken together, these diagnostics indicate that hierarchical clustering identifies a similar four-cluster structure to the one recovered by K-Means. Although the method exhibits slightly lower silhouette scores, the merge patterns and dendrogram topology confirm that the dataset contains four meaningful and interpretable groups.



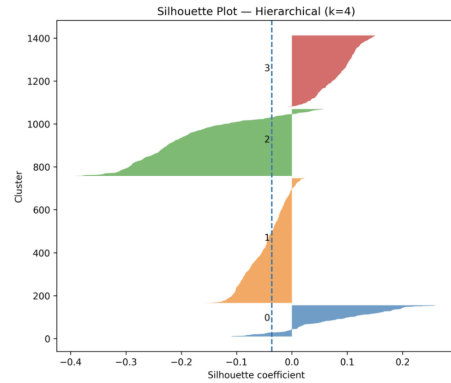
(a) Last 50 merge distances (Ward).



(b) Truncated dendrogram (Ward).



(c) Silhouette scores across k .



(d) Silhouette distribution for $k = 4$.

Figure 9: Hierarchical clustering diagnostics using Ward linkage.

4.2.4 K-Medoids (PAM)

K-Medoids clustering was applied as a robustness check against the centroid-based behavior of K-Means. Unlike K-Means, which computes synthetic centroids, K-Medoids selects actual observations (medoids) as cluster centers, making it more resilient to noise and extreme values—an important consideration given the heavy-tailed engagement variables in the dataset.

To determine the appropriate number of clusters, we evaluated both the PAM Elbow curve and the Silhouette trend. The Elbow curve (Figure 10a) shows a clear decline in the sum of distances between $k = 2$ and $k = 4$, followed by a flattening that indicates diminishing returns. The Silhouette scores (Figure 10b) similarly decrease sharply after $k = 2$ but stabilize around $k = 4$, suggesting that four clusters offer a reasonable balance between separation and cohesion.

Using $k = 4$, we visualized the resulting clusters via a PCA projection. The 2D PCA plot (Figure 10c) shows four separable groups whose spatial arrangement closely mirrors the structure identified by K-Means. This consistency across fundamentally different clustering algorithms reinforces the stability of the four-segment solution.

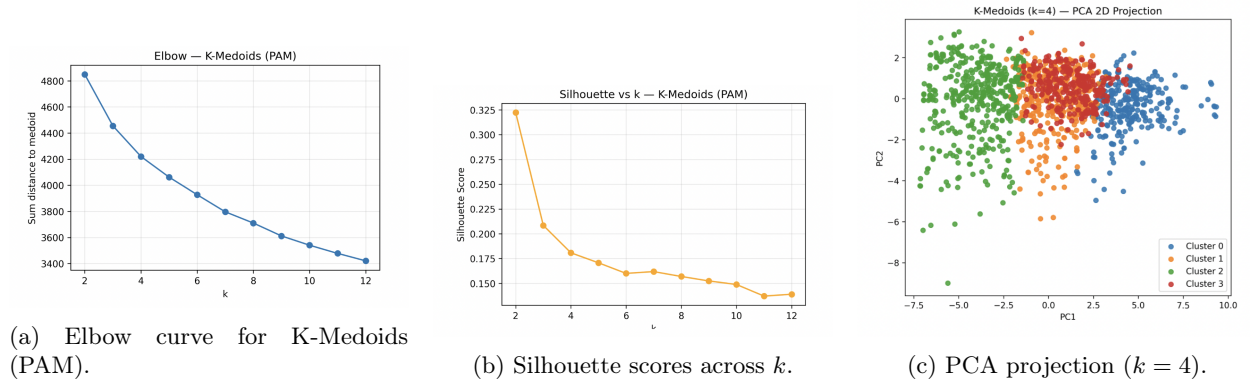


Figure 10: K-Medoids diagnostics: elbow curve, silhouette scores, and PCA visualization.

4.2.5 Clustering Summary / Interpretation

Across all three algorithms, a consistent four-cluster structure emerged. Each cluster captured a distinct combination of price, engagement intensity, and rating patterns, forming stable and interpretable product segments. This consistency across methods suggests that the underlying feature space contains a natural segmentation, which provides a strong foundation for comparing clusters with subsequent sentiment patterns and brand-level differences.

4.3 Sentiment Analysis Results (BERT)

4.3.1 Sentiment Analysis: Exploratory Data Analysis

Before applying the BERT-based sentiment classifier, we conducted an exploratory analysis of the textual review data to understand how linguistic patterns varied across rating levels and sentiment polarity. This step helped determine whether simple textual cues, such as review length, were meaningfully associated with consumer sentiment, and whether preprocessing or class-balancing considerations were necessary for the modeling stage.

We first examined the distribution of review lengths across the five star-rating categories. As shown in Figure 11a, reviews of all ratings exhibited a right-skewed distribution, with most reviews falling under 100 words but with a long tail of substantially longer entries. Although median review lengths were similar across rating levels, the interquartile ranges expanded slightly for higher ratings, suggesting that positive reviews tend to contain more detailed descriptions or personal experiences. However, the overlap between

categories indicates that review length alone is not sufficient for distinguishing sentiment, justifying the need for a more advanced text-based classifier.

Next, we compared review lengths for positive and negative reviews after grouping them into binary sentiment categories. Figure 11b shows nearly identical length distributions between positive and negative reviews, with both following the same heavy-tailed pattern. This similarity confirms that review length is not a reliable predictor of sentiment and reinforces the importance of semantic modeling approaches such as BERT.

Finally, Figure 11c shows the overall distribution of review lengths across the full dataset. The concentration of shorter reviews combined with the long tail of highly detailed entries highlights the variability of user expression in beauty product reviews. This variability has direct modeling implications: preprocessing must be robust to inconsistent text lengths, and the sentiment model must capture meaning across both short and extended reviews.

Taken together, these EDA findings emphasize the need for a contextual language model rather than shallow sentiment heuristics. Length alone does not meaningfully differentiate sentiment categories, and the diversity of textual patterns supports the selection of a transformer-based classifier for more accurate sentiment prediction.

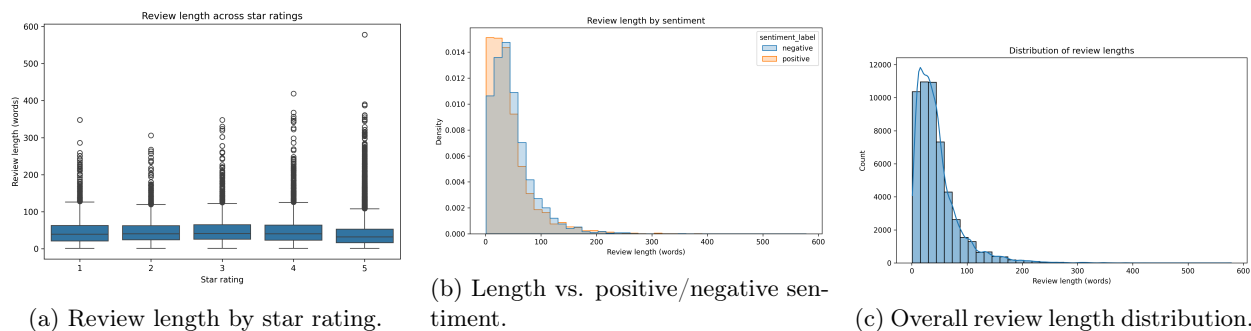


Figure 11: Review length patterns across ratings, sentiment categories, and overall distributions.

4.3.2 Word-level patterns

To understand how consumers linguistically express satisfaction and dissatisfaction, we performed a word-level comparison between positive and negative reviews. The two word clouds in Figure 12 highlight clear contrasts in descriptive focus and emotional tone across the two sentiment groups.

Positive reviews frequently emphasized terms such as “skin”, “product”, “use”, “love”, “foundation”, “color”, “smooth”, “natural”, and “blend”. These words suggest that satisfied users often highlight how a product feels on the skin, how seamlessly it blends, how natural or even the finish appears, and how well it performs during daily use. Many of these terms also correspond with attributes emphasized in earlier EDA findings, where high-rated products tended to be associated with descriptors related to texture, wearability, and ease of application. For example, in the EDA word cloud for positive reviews, large and prominent terms like “skin”, “product”, and “foundation” indicate that customers strongly focus on complexion-related results when expressing satisfaction. Additionally, positive reviews include verbs such as “use”, “wear”, “apply”, and “blend”, reflecting smooth application and reliable everyday performance as central aspects of positive user experience.

Negative reviews, by contrast, contain a markedly different set of dominant terms, including “dry”, “oily”, “patchy”, “cakey”, “shade”, “tone”, “powder”, “work”, and “tried”. These words indicate common points of frustration: texture inconsistency (e.g., dryness, oiliness, patchiness), poor shade matching, and products that fail to perform reliably during application or wear. The strong presence of words such as “look”, “make”, “face”, and “work” in negative reviews underscores that dissatisfaction is often grounded in the final appearance on the skin or issues encountered during application. Additionally, many negative reviews

views of top-rated products, underscoring how finish, comfort, and visual outcome shape highly positive sentiment.

In contrast, products with lower predicted sentiment (average sentiment ≤ 2) generally show recurring themes of dissatisfaction associated with texture inconsistency, shade mismatch, insufficient coverage, or poor wear time. Negative reviews commonly include terms such as “dry”, “oily”, “patchy”, “cakey”, and “oxidized”, emphasizing problematic sensory and appearance outcomes. These sentiment patterns mirror the EDA results highlighting the lowest-rated products (Figures ??), many of which experienced consistent complaints about dryness, uneven application, or foundation shades running too light or too dark. Text-based sentiment analysis captures these same issues, but with far greater nuance, as reviewers frequently specify *why* a product failed for them—for example, by noting that a concealer accentuated fine lines or a foundation oxidized after application.

An important observation is that some products with relatively few reviews display unusually high or low sentiment scores. This volatility reflects the right-skewed review distribution identified in the EDA: products with sparse review activity are more sensitive to the tone of each individual review. A single highly positive or negative comment can dramatically influence the average predicted sentiment. By contrast, high-volume products, such as those in the “popular” clusters identified by K-Means and K-Medoids, show far more stable sentiment profiles due to the large number of textual data points smoothing out individual extremes.

Taken together, the product-level sentiment analysis demonstrates that positive consumer experiences are strongly associated with smooth application, high blendability, reliable coverage, and a natural-looking finish, while negative experiences stem from inconsistency, mismatch, or sensory discomfort. These insights highlight the value of sentiment scores not only as descriptive indicators of customer satisfaction but also as diagnostic signals that identify precisely which product-level attributes drive satisfaction or frustration. By linking text-based sentiment to the clustering and EDA findings, this analysis provides a comprehensive understanding of how individual products perform within the broader competitive landscape of cosmetic offerings.

Most-mentioned words for top-rated products (circular view)

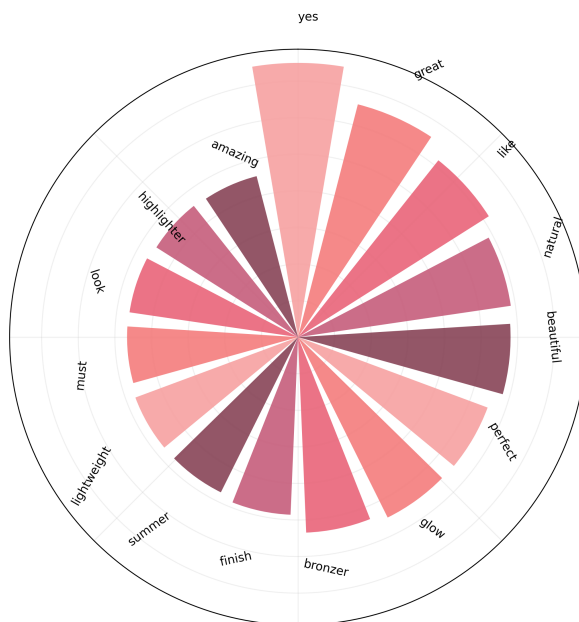


Figure 13: Most frequent words in reviews for top-rated products visualized in a circular radial format.

4.3.4 Brand-level sentiment

At the brand level, the aggregated sentiment scores reveal broader patterns in how consumers perceive overall product lines rather than individual items. While product-level sentiment captures the specific attributes that drive positive or negative experiences, brand-level sentiment reflects the consistency, reputation, and reliability that consumers associate with each brand’s offerings.

As shown in Figure 14, the highest-rated brands tend to be those that deliver consistent formulation quality across multiple product categories, including complexion, eye, and lip products. Brands such as Persona, Hey Honey, and *r.e.m. beauty* exhibit strong average sentiment scores, suggesting that users view their products as reliable and high-performing across a range of use cases. These brands frequently appear in positive reviews that highlight smooth application, natural finish, long wear, and comfortable texture—patterns that align closely with the word-level insights observed earlier.

In contrast, brands with lower average sentiment scores typically show patterns of user dissatisfaction related to inconsistent shade ranges, dry or cakey textures, limited longevity, or mismatched marketing claims. These issues are often cited across multiple products rather than isolated to a single item, indicating systemic formulation or positioning challenges. The consistency of these negative themes in the catalog of a brand suggests that the brand-level sentiment can reveal deeper structural issues that are not always visible when analyzing products individually.

Brand-level findings complement product-level analysis by illustrating how sentiment patterns scale beyond individual items to shape overall consumer trust. Together, these insights provide a multi-layered understanding of user preferences across both product and brand dimensions in the cosmetic market.

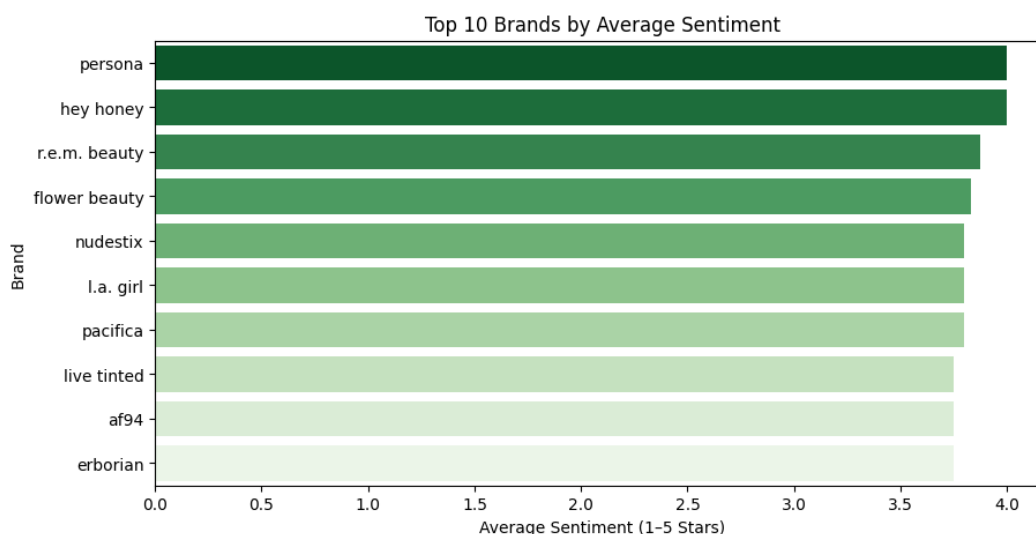


Figure 14: Top brands by average predicted sentiment.

4.3.5 Overall insight

Taken together, sentiment analysis from word-, product-, and brand-level perspectives shows a coherent and multidimensional view of consumer preferences in the cosmetics market. At all levels of analysis, positive sentiment is strongly related to product performance factors such as smooth application, natural finish, reliable coverage, and long-lasting wear. These themes appear not only in the most frequent Words found in highly rated reviews but also in the products and brands that consistently achieve the highest sentiment averages.

On the other hand, negative sentiment focuses on recurring issues like dryness, patchiness, cakiness, shade mismatch, oxidation, and short wear time. These complaints surface time and again throughout the

poorly rated Products range from high-end to more affordable and encompass brands lacking in formulation consistency or limited shade inclusivity. The Alignment between textual complaints and low predicted sentiment scores would suggest that the model effectively:

Captures the drivers underlying user dissatisfaction. Another key pattern to emerge from this study is that of consistency. The best brands generally exhibit consistent Quality across several product categories ensures stable positive sentiment while lower-performing

Brands show sentiment patterns reflective of broader formulation or positioning weaknesses. This distinction illustrates how product-level variation aggregates into brand-level reputation. Finally, the strong correspondence between EDA findings, clustering insights, and sentiment model outputs. indicates that consumer experiences are shaped by a clear set of functional and sensory attributes. The combined analysis thus yields a holistic understanding of how individual product characteristics, brand Identity and user expectations interact in influencing overall satisfaction. This combined view points to several areas where products could be improved, especially in shade range, texture, and long-wear performance, all of which would noticeably elevate how users experience and perceive these cosmetics.

5 Discussion

The findings of this study provide a comprehensive understanding of how makeup products vary in popularity, performance, and consumer satisfaction across the Ulta marketplace. By combining clustering, exploratory data analysis, and sentiment modeling, the results offer a multi layered view of consumer behavior that highlights both strengths and weaknesses in product performance.

Across all three clustering methods, a consistent pattern of four product segments emerged. This agreement across different algorithms suggests that the structure of the product space is stable and meaningful. Products varied along dimensions such as engagement intensity, pricing, and star rating patterns. Engagement related variables, including total reviews, native reviews, and syndicated reviews, consistently formed a strong correlated group, reflecting their shared role as indicators of visibility and consumer interest. In contrast, attributes such as price, rating, and shade count showed weaker associations with these engagement metrics, which reinforces the idea that popularity is not solely determined by inherent product features. These patterns support the usefulness of clustering as a tool for identifying meaningful segments of the beauty market.

The sentiment analysis offered additional insight into how consumers describe their experiences. At the word level, positive reviews focused on smooth textures, natural finishes, blendability, and long wear time. Negative reviews frequently mentioned dryness, patchiness, cakiness, oxidation, and shade mismatch. These patterns indicate that texture and finish are central to positive user experience, while inconsistency in formula or shade matching is a significant source of dissatisfaction. These findings align with earlier exploratory patterns that showed strong variability in star ratings even among products with similar price points or brand affiliations. They also confirm that the BERT model effectively captured the underlying drivers of satisfaction and frustration.

At the product and brand levels, the sentiment results show that consistent performance is one of the most important factors shaping consumer trust. High performing brands maintained stable positive sentiment across multiple product categories, while brands with lower sentiment often exhibited recurring formulation issues. This consistency suggests that brand reputation in the beauty space is shaped collectively by the quality of several products rather than individual successes or failures. These findings further demonstrate the value of combining numerical clustering with text based analysis, as the two approaches reinforce each other and reveal deeper patterns that would not be apparent from one method alone.

Overall, the integrated methods used in this study show that consumer behavior in the cosmetics industry is influenced by a clear set of product characteristics, emotional reactions, and engagement patterns. Understanding these relationships can support more informed product development and more effective marketing strategies while identifying areas for improvement in texture, shade variety, and formula reliability.

6 Conclusion

This study used clustering methods and sentiment analysis to examine consumer behavior, product performance, and brand level patterns in the Ulta Beauty makeup market. By analyzing product metadata and more than 300,000 customer reviews, the results revealed four clear product segments that consistently appeared in K Means, Hierarchical Clustering, and K Medoids. These groups reflected meaningful differences in engagement intensity, pricing, and rating behavior, indicating that the marketplace contains natural divisions that reflect how consumers interact with makeup products.

The sentiment analysis using a BERT based model added an important layer of understanding by identifying the emotional tone of customer reviews. Positive sentiment centered on smooth application, natural finishes, and reliable wear, while negative sentiment focused on dryness, patchiness, shade mismatch, and inconsistent performance. These patterns demonstrate that consumers value both functional quality and sensory experience in makeup products.

The combined findings reveal that consumer satisfaction is shaped by a blend of product attributes, brand consistency, and emotional responses. These insights can guide future product improvements, enhance customer experience strategies, and support the development of tools such as recommendation systems or brand level performance dashboards. The integration of clustering and sentiment modeling provides a strong foundation for understanding the rapidly evolving beauty industry and highlights opportunities for continued innovation in product design and analytics.

References

- [1] M. A. Al Khaldy, B. A. A. Al-Obaydi, and A. J. Al Shari. The impact of predictive analytics and ai on digital marketing strategy and roi. *ResearchGate*, 3(653), 2023.
- [2] S. Arora, V. A. Athavale, H. Maggu, and A. Agarwal. Artificial intelligence and virtual assistant—working model. *Mobile Radio Communications and 5G Networks: Proceedings of MRCN 2020*, pages 163 – 171, 2020.
- [3] T. Chen, P. Samaranayake, X. Cen, M. Qi, and Y.-C. Lan. The impact of online reviews on consumers’ purchasing decisions: Evidence from an eye-tracking study. *Frontiers in Psychology*, 13, 2022.
- [4] . K. K. L. Kotler, P. *Marketing management (Global Edition)*, 2016.
- [5] H. Masuda, S. H. Han, and J. Lee. Impacts of influencer attributes on purchase intentions in social media influencer marketing: Mediating roles of character trust and brand trust. *Technological Forecasting and Social Change*, 174, 2022.
- [6] N. Rezki, M. Mansouri, and R. Oucheikh. Analysis of consumer satisfaction using clustering methods. *Management Systems in Production Engineering*, 33(1):72–81, 2025.

A Appendix A: Missing Value Patterns and Heatmaps Code

A.1 A.1 Notebook Setup and Data Loading

```
# -*- coding: utf-8 -*-
"""capstone final draft_missing value pattern heatmaps.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1b_cbpPj9rse1yd8yUdzIhRubZIONsDAN
"""
```

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

reviews = pd.read_csv("cleaned_makeup_reviews.csv")
products = pd.read_csv("cleaned_makeup_products.csv")

import nltk
nltk.download('stopwords')

```

A.2 A.2 Additional EDA

```

plt.figure(figsize=(12, 6))
sns.heatmap(reviews.isnull(), cbar=False, yticklabels=False)
plt.title("Missing Value Pattern - Reviews Dataset")
plt.tight_layout()
plt.savefig("missing_pattern_reviews.png", dpi=300)
plt.close()

plt.figure(figsize=(12, 6))
sns.heatmap(products.isnull(), cbar=False, yticklabels=False)
plt.title("Missing Value Pattern - Products Dataset")
plt.tight_layout()
plt.savefig("missing_pattern_products.png", dpi=300)
plt.close()

print("Heatmaps saved: missing_pattern_reviews.png, missing_pattern_products.png")

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

def clean_and_scale_dataset(df, skewed_cols=None, id_cols=None):
    """
    df:          DataFrame
    skewed_cols: log1p                                (ex: ['num_reviews', '
                helpful_votes', ...])
    id_cols:      /                                    (ex: ['
                product_id', 'brand'])
    """
    df = df.copy()

    if id_cols is None:
        id_cols = []

    # 1.                                m e d i a n
    numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
    numeric_cols = [c for c in numeric_cols if c not in id_cols]

    df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median())

    # 2.                                (variance 0)
    constant_cols = [c for c in numeric_cols if df[c].nunique() <= 1]

```

```

if constant_cols:
    print("Dropping constant columns:", constant_cols)
    df.drop(columns=constant_cols, inplace=True)
    numeric_cols = [c for c in numeric_cols if c not in constant_cols]

# 3.
# (log1p skewed_cols)
if skewed_cols is not None:
    #
    skewed_cols = [c for c in skewed_cols if c in df.columns]
    for col in skewed_cols:
        # (engagement)
        df[col] = np.log1p(df[col])
        print(f"Applied log1p to: {col}")

# 4. StandardScaler
numeric_cols = [c for c in df.select_dtypes(include=[np.number]).columns if c
                not in id_cols]

scaler = StandardScaler()
df_scaled = df.copy()
df_scaled[numeric_cols] = scaler.fit_transform(df[numeric_cols])

print("Scaled numeric columns:", numeric_cols)

return df_scaled, scaler

# numeric (
)
candidate_cols = [
    "price",
    "num_shades",
    "rating", # average rating
    "num_reviews", # total review count
    "helpful_votes", # helpfulness metric
    "native_review_count",
    "syndicated_review_count"
]

#
numeric_cols = [c for c in candidate_cols if c in products.columns]
corr_df = products[numeric_cols].dropna()

print("Using columns for correlation:", numeric_cols)

# 1. pairplot (/ )
sns.pairplot(corr_df[numeric_cols])
plt.suptitle("Pairwise relationships among key product features", y=1.02)
plt.tight_layout()
plt.savefig("products_pairplot_correlation.png", dpi=300)
plt.show()
plt.close()

from wordcloud import WordCloud
from sklearn.feature_extraction.text import TfidfVectorizer

```

```

# -----
# /
# -----
CSV_PATH = "cleaned_makeup_reviews.csv"

TEXT_COL = "comments" #
RATING_COL = "rating" # (1~5)
SENT_SCORE_COL = "sentiment_score" # BERT 1~5
SENT_LABEL_COL = "sentiment_label" # "positive"/"negative" (
)

# -----
#
# -----
df = pd.read_csv(CSV_PATH)

#
df = df.dropna(subset=[TEXT_COL])

# sentiment_label sentiment_score
if SENT_LABEL_COL not in df.columns and SENT_SCORE_COL in df.columns:
    df[SENT_LABEL_COL] = np.where(df[SENT_SCORE_COL] >= 3, "positive", "negative")

print("Data shape after cleaning:", df.shape)
print("Columns:", df.columns)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
from collections import Counter
from wordcloud import WordCloud

# Ensure text column is string
df["comments"] = df["comments"].astype(str)

# Create review_length (
df["review_length"] = df["comments"].str.split().str.len()

# Create simple sentiment label based on rating
def label_from_rating(r):
    if r >= 4:
        return "positive"
    elif r <= 2:
        return "negative"
    else:
        return "neutral"

df["sentiment_label"] = df["rating"].apply(label_from_rating)

df.head()

plt.figure(figsize=(8,5))
sns.histplot(df["review_length"], bins=40, kde=True)
plt.xlabel("Review length (words)")

```

```

plt.title("Distribution of review lengths")
plt.tight_layout()
plt.savefig("sent_eda_review_length_overall.png", dpi=300, bbox_inches="tight")
plt.show()
plt.close()

df_pn = df[df["sentiment_label"].isin(["positive", "negative"])]

plt.figure(figsize=(8,5))
sns.histplot(data=df_pn, x="review_length", hue="sentiment_label",
             bins=40, element="step", stat="density", common_norm=False)
plt.xlabel("Review length (words)")
plt.title("Review length by sentiment")
plt.tight_layout()
plt.savefig("sent_eda_review_length_by_sentiment.png", dpi=300, bbox_inches="tight")
plt.show()
plt.close()

plt.figure(figsize=(7,5))
sns.boxplot(data=df, x="rating", y="review_length")
plt.xlabel("Star rating")
plt.ylabel("Review length (words)")
plt.title("Review length across star ratings")
plt.tight_layout()
plt.savefig("sent_eda_rating_vs_length_boxplot.png", dpi=300, bbox_inches="tight")
plt.show()
plt.close()

reviews = pd.read_csv("cleaned_makeup_reviews.csv")
print(reviews.columns)

products = pd.read_csv("cleaned_makeup_products.csv")
print(products.columns)

import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
import re
import nltk
from nltk.corpus import stopwords

reviews = pd.read_csv("cleaned_makeup_reviews.csv")
products = pd.read_csv("cleaned_makeup_products.csv")

print("Reviews columns:", reviews.columns.tolist())
print("Products columns:", products.columns.tolist())

MIN_REVIEWS = 50
TOP_N = 10

top_products = (
    products
    .query("review_count >= @MIN_REVIEWS")
    .sort_values(["average_rating", "review_count"], ascending=[False, False])
    .head(TOP_N)

```

```

        .copy()
    )

    print("\nTop products by average_rating:")
    print(top_products[["product_name", "brand", "average_rating", "review_count"]])

    top_ids = top_products["product_link_id"].unique()

    top_reviews = reviews[reviews["product_link_id"].isin(top_ids)].copy()

    for col in ["headline", "comments", "bottom_line"]:
        if col not in top_reviews.columns:
            top_reviews[col] = ""

    top_reviews["full_text"] = (
        top_reviews["headline"].fillna("").astype(str) + " " +
        top_reviews["comments"].fillna("").astype(str) + " " +
        top_reviews["bottom_line"].fillna("").astype(str)
    )

    eng_stop = set(stopwords.words("english"))

    custom_stop = {
        "product", "makeup", "foundation", "primer", "concealer",
        "shade", "color", "skin", "face", "use", "used", "love",
        "one", "really"
    }

    def clean_tokenize(text: str):
        text = text.lower()
        text = re.sub(r"[^a-z\s]", " ", text)
        tokens = text.split()
        tokens = [
            t for t in tokens
            if t not in eng_stop
            and t not in custom_stop
            and len(t) > 2
        ]
        return tokens

    all_tokens = []
    for txt in top_reviews["full_text"].astype(str):
        all_tokens.extend(clean_tokenize(txt))

    word_counts = Counter(all_tokens)

    TOP_K_WORDS = 15
    most_common = word_counts.most_common(TOP_K_WORDS)

    if not most_common:
        print("No tokens found      check text columns or stopwords list!")
    else:
        words, freqs = zip(*most_common)

        plt.figure(figsize=(8, 5))
        plt.barh(words[::-1], freqs[::-1])

```

```

plt.gca().invert_yaxis()
plt.xlabel("Frequency")
plt.title("Most-mentioned words in reviews for top-rated products")
plt.tight_layout()

plt.savefig("most_mentioned_words_top_products.png", dpi=300)
plt.show()

print("\nSaved figure: most_mentioned_words_top_products.png")

plt.figure(figsize=(8, 8))
ax = plt.subplot(111, polar=True)

bars = ax.bar(
    angles,
    radii,
    width=2 * np.pi / N * 0.8,
    bottom=0.0,
    color=bar_colors,
    alpha=0.9
)

ax.set_axisbelow(True)
ax.grid(alpha=0.2)
ax.set_theta_zero_location("N")
ax.set_theta_direction(-1)
ax.set_yticklabels([])
ax.set_xticklabels([])

for angle, radius, word in zip(angles, radii, words):
    label_radius = radius + 0.25
    rotation = np.degrees(angle)

    if 90 < rotation < 270:
        rotation = rotation + 180
        alignment = "right"
    else:
        alignment = "left"

    ax.text(
        angle,
        label_radius,
        word,
        ha=alignment,
        va="center",
        rotation=rotation,
        rotation_mode="anchor",
        fontsize=10
    )

plt.title(
    "Most-mentioned words for top-rated products (circular view)",
    pad=60,
    fontsize=16
)

```

```

plt.tight_layout()
plt.savefig("most_mentioned_words_circle_paletteA.png", dpi=300)
plt.show()

import matplotlib.pyplot as plt
import numpy as np

N = len(words)
freqs_arr = np.array(freqs, dtype=float)

sizes = (freqs_arr / freqs_arr.max()) * 2500 + 500

cols = int(np.ceil(np.sqrt(N)))
rows = int(np.ceil(N / cols))

xs = []
ys = []
for i in range(N):
    r = i // cols
    c = i % cols
    xs.append(c + np.random.uniform(-0.15, 0.15))
    ys.append(-r + np.random.uniform(-0.15, 0.15))

xs = np.array(xs)
ys = np.array(ys)

bubble_colors = [PALETTE_A[i % len(PALETTE_A)] for i in range(N)]

plt.figure(figsize=(8, 6))
plt.scatter(xs, ys, s=sizes, c=bubble_colors, alpha=0.85)

for x, y, word in zip(xs, ys, words):
    plt.text(
        x,
        y,
        word,
        ha="center",
        va="center",
        fontsize=9,
        color="#FFFFFF",
        weight="bold"
    )

plt.axis("off")
plt.title("Most-mentioned words for top-rated products (bubble view)", pad=15)

plt.tight_layout()
plt.savefig("most_mentioned_words_bubble_paletteA.png", dpi=300)
plt.show()

import matplotlib.pyplot as plt
import numpy as np

PALETTE_A = ["#F7A1A1", "#F47C7C", "#E96479", "#C65D7B", "#874356"]

N = len(words)

```



```

angles = np.linspace(0, 2 * np.pi, N, endpoint=False)

freqs_arr = np.array(freqs, dtype=float)
norm = (freqs_arr - freqs_arr.min()) / (freqs_arr.max() - freqs_arr.min() + 1e-9)
lengths = 0.4 + norm * 0.8

circle_colors = [PALETTE_A[i % len(PALETTE_A)] for i in range(N)]

plt.figure(figsize=(9, 9))
ax = plt.subplot(111, polar=True)

for angle, L in zip(angles, lengths):
    ax.plot([angle, angle], [0.1, L], color="#E96479", linewidth=2.0, alpha=0.7)

ax.scatter(
    angles,
    lengths,
    c=circle_colors,
    s=250,
    alpha=0.95,
    edgecolors="#FFFFFF",
    linewidths=1.5
)

for angle, L, word in zip(angles, lengths, words):
    label_r = L + 0.12
    rotation = np.degrees(angle)

    if 90 < rotation < 270:
        rotation = rotation + 180
        alignment = "right"
    else:
        alignment = "left"

    ax.text(
        angle,
        label_r,
        word,
        ha=alignment,
        va="center",
        rotation=rotation,
        rotation_mode="anchor",
        fontsize=9
    )

ax.set_xticklabels([])
ax.set_yticklabels([])
ax.grid(alpha=0.15)

ax.set_theta_zero_location("N")
ax.set_theta_direction(-1)

plt.title("Most-mentioned words for top-rated products (Polar Lollipop Chart)",
    pad=30)
plt.tight_layout()

```

```
plt.savefig("most_mentioned_words_polar_lollipop_paletteA.png", dpi=300)
plt.show()
```

B Appendix B: Full Methodology and Modeling Code

B.1 B.1 Clustering, PCA, and Exploratory Analysis

```
# -*- coding: utf-8 -*-
"""Capstone Methodology.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1_1alihTmdB4w5V0cXLjhD1bbYib3VpAh
"""

# 1      Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

"""--- Clustering ---"""

import pandas as pd

# Path to your Drive folder
base = "/content/drive/MyDrive/Dat Capstone"

# Load both CSVs
products = pd.read_csv(f"{base}/cleaned_makeup_products.csv")
reviews = pd.read_csv(f"{base}/cleaned_makeup_reviews.csv")

# Check shapes and preview
print("Products shape:", products.shape)
print("Reviews shape:", reviews.shape)

display(products.head())
display(reviews.head())

# --- Core ---
import pandas as pd
import numpy as np

# --- Modeling & evaluation ---
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score, silhouette_samples

# --- Utils & plots ---
import matplotlib.pyplot as plt
import textwrap
import os
from pathlib import Path
np.random.seed(42)
```

```

base = "/content/drive/MyDrive/Dat Capstone"          # <-- change if needed
csv_path = f"{base}/cleaned_makeup_products.csv"

# low_memory=False avoids the mixed dtype warning, and we'll coerce numeric below
df = pd.read_csv(csv_path, low_memory=False)

print("Shape:", df.shape)
display(df.head(3))

# optional: peek at dtypes to see text vs numeric
df.info()

# 2) define columns & clean (IDs/text dropped, numerics kept)      SAFE

expected_cols = [
    "product_link_id", "product_link", "category", "item_id", "product_name", "brand", "
    price", "num_shades",
    "rating", "num_reviews", "description", "pros", "cons", "best_uses", "
    describe_yourself",
    "review_star_1", "review_star_2", "review_star_3", "review_star_4", "review_star_5
    ",
    "rating_star_1", "rating_star_2", "rating_star_3", "rating_star_4", "rating_star_5
    ",
    "rating_count", "review_count", "average_rating", "recommended_ratio",
    "native_review_count", "native_sampling_review_count",
    # "native_community_count",    # <-- not present in your file; comment out or
    let code skip missing
    "syndicated_review_count", "faceoff_negative", "faceoff_positive"
]

present_cols = [c for c in expected_cols if c in df.columns]
missing_cols = sorted(set(expected_cols) - set(df.columns))
print("Missing (ignored):", missing_cols)

# drop ID/url/text/meta for clustering (keep for reporting later)
drop_cols = [
    "product_link_id", "product_link", "item_id", "product_name",
    "description", "pros", "cons", "best_uses", "describe_yourself", "brand", "category"
]

drop_cols = [c for c in drop_cols if c in df.columns] # keep only those that
exist

# candidate numeric features (that actually exist)
num_cols = [c for c in present_cols if c not in drop_cols]
print("Candidate numeric columns:", num_cols)

# coerce everything in num_cols to numeric where possible
for c in num_cols:
    df[c] = pd.to_numeric(df[c], errors="coerce")

numeric_df = df.loc[:, num_cols].copy()
numeric_df = numeric_df.dropna(axis=1, how="all")

print("Numeric columns retained:", list(numeric_df.columns))
display(numeric_df.describe().T.round(3).head(15))

```

```

import math
import matplotlib.pyplot as plt

cols = numeric_df.columns
n = len(cols)
plots_per_fig = 4 # 2x2 layout      4 plots per figure

for start in range(0, n, plots_per_fig):
    end = min(start + plots_per_fig, n)
    subset = cols[start:end]

    nrows, ncols = 2, 2
    fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(12, 8))
    axes = axes.flatten()

    for i, c in enumerate(subset):
        ax = axes[i]
        ax.hist(numeric_df[c].dropna(), bins=30, color="skyblue", edgecolor="black")
        ax.set_title(f"Histogram: {c}", fontsize=11)
        ax.set_xlabel(c)
        ax.set_ylabel("Count")

    # hide unused subplots if fewer than 4 left
    for j in range(i+1, len(axes)):
        axes[j].set_visible(False)

    plt.tight_layout()
    fig_num = start // plots_per_fig + 1
    plt.savefig(f"{base}/basic_eda_part{fig_num}.png", dpi=160, bbox_inches="tight")
    plt.show()

# Step 4: Handle missing values

X = numeric_df.copy()
X = X.fillna(X.median(numeric_only=True))

# drop columns with no variance (e.g., same value for all)
low_var = X.nunique() <= 1
if low_var.any():
    print("Dropping near-constant columns:", list(X.columns[low_var]))
    X = X.loc[:, ~low_var]

# Step 5: Log-transform skewed variables
import numpy as np

skew_candidates = [c for c in X.columns if (X[c].max() > 100) and (X[c].min() >= 0)]
print("Skew candidates (log1p transform):", skew_candidates)

X_log = X.copy()
for c in skew_candidates:
    X_log[c] = np.log1p(X[c]) # log1p handles zeros

# Step 6: Standardize data (scaling)

```

```

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X_log), columns=X_log.columns, index=
    X_log.index)

display(X_scaled.head(3))

#Step 7: Elbow method
from sklearn.cluster import KMeans

inertias = []
ks = range(2, 13)

for k in ks:
    km = KMeans(n_clusters=k, n_init=20, random_state=42)
    km.fit(X_scaled)
    inertias.append(km.inertia_)

plt.figure(figsize=(6,4))
plt.plot(list(ks), inertias, marker="o")
plt.xlabel("k")
plt.ylabel("Within-Cluster Sum of Squares (Inertia)")
plt.title("Elbow Curve for K-Means")
plt.grid(True, alpha=.3)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/elbow_curve_for_kmeans.png",
    dpi=300, bbox_inches="tight")
plt.show()

for k, s in zip(ks, inertias):
    print(f"k={k:2d}    inertia={s:,.0f}")

#Step 8: Silhouette scores

from sklearn.metrics import silhouette_score

sil_scores = []
for k in ks:
    km = KMeans(n_clusters=k, n_init=20, random_state=42)
    labels = km.fit_predict(X_scaled)
    sil = silhouette_score(X_scaled, labels)
    sil_scores.append(sil)

plt.figure(figsize=(6,4))
plt.plot(list(ks), sil_scores, marker="o")
plt.xlabel("k")
plt.ylabel("Silhouette Score")
plt.title("Silhouette vs k")
plt.grid(True, alpha=.3)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/silhouette_vs_k.png",
    dpi=300, bbox_inches="tight")
plt.show()

```

```

for k, s in zip(ks, sil_scores):
    print(f"k={k:2d}    silhouette={s:.3f}")

# Choose the best number of clusters based on your plots
k_best = 4 # <-- change this if another value looks better

from sklearn.cluster import KMeans

# Fit K-Means with chosen k
km_final = KMeans(n_clusters=k_best, n_init=50, random_state=42)
labels = km_final.fit_predict(X_scaled)

# Attach labels to original dataframe
df_clusters = df.copy()
df_clusters["cluster"] = labels

print("Cluster sizes:")
print(df_clusters["cluster"].value_counts().sort_index())

# quick look
display(df_clusters[["product_name", "brand", "price", "average_rating", "cluster"]
        ].head(10))

# STEP 9: Final K-Means model
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_samples
import matplotlib.pyplot as plt
import numpy as np

k_best = 4 # final chosen k
km_final = KMeans(n_clusters=k_best, n_init=50, random_state=42)
labels = km_final.fit_predict(X_scaled)

df_clusters = df.copy()
df_clusters["cluster"] = labels

print("Cluster sizes:")
print(df_clusters["cluster"].value_counts().sort_index())

# Step 10      PCA 2D visualization
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

pca = PCA(n_components=2, random_state=42)
proj2 = pca.fit_transform(X_scaled)

plt.figure(figsize=(7,6))
for c in range(k_best):
    idx = (labels == c)
    plt.scatter(proj2[idx, 0], proj2[idx, 1], s=25, alpha=.85, label=f"Cluster {c}")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title(f"K-Means (k={k_best})      PCA 2D Projection")
plt.legend()

```

```

plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/pca_2d_projection.png",
            dpi=300, bbox_inches="tight")
plt.show()

print("Explained variance ratio:", pca.explained_variance_ratio_)

from sklearn.metrics import silhouette_samples
import numpy as np
import matplotlib.pyplot as plt

sample_sil = silhouette_samples(X_scaled, labels)

y_lower = 10
plt.figure(figsize=(7,6))
for i in range(k_best):
    ith = sample_sil[labels == i]
    ith.sort()
    size_i = ith.shape[0]
    y_upper = y_lower + size_i
    plt.fill_betweenx(np.arange(y_lower, y_upper), 0, ith, alpha=0.7)
    plt.text(-0.05, y_lower + 0.5*size_i, str(i))
    y_lower = y_upper + 10

plt.axvline(np.mean(sample_sil), linestyle="--")
plt.xlabel("Silhouette coefficient")
plt.ylabel("Cluster")
plt.title(f"Silhouette Plot      k={k_best}")
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/silhouette_plot_k-4vs_k.png",
            dpi=300, bbox_inches="tight")
plt.show()

import matplotlib.pyplot as plt
import math

# assuming you already have X_scaled or your feature data
kmeans = KMeans(n_clusters=k_best, random_state=42)
kmeans.fit(X_scaled)

# convert cluster centers to a DataFrame
centers_orig = pd.DataFrame(kmeans.cluster_centers_, columns=X_scaled.columns)
out_dir = Path(".") # current directory

# set how many plots per page
cols_per_fig = 3
rows_per_fig = 3
plots_per_fig = cols_per_fig * rows_per_fig

cols = centers_orig.columns.tolist()
num_plots = len(cols)
num_figs = math.ceil(num_plots / plots_per_fig)

for f in range(num_figs):
    start = f * plots_per_fig
    end = start + plots_per_fig

```

```

subset = cols[start:end]

fig, axes = plt.subplots(rows_per_fig, cols_per_fig, figsize=(12, 9))
axes = axes.flatten()

for i, col in enumerate(subset):
    axes[i].bar(centers_orig.index, centers_orig[col].values)
    axes[i].set_title(col, fontsize=10)
    axes[i].tick_params(axis='x', rotation=30)
    axes[i].set_ylabel("")
# turn off unused subplots
for j in range(len(subset), len(axes)):
    axes[j].axis("off")

plt.tight_layout()
plt.savefig(out_dir / f"centers_grid_summary_{f+1}.png", dpi=160)
plt.show()

# Use the exact features that went into modeling
features_used = list(X.columns)

profile_count = df_clusters["cluster"].value_counts().sort_index().rename("count")
profile_mean = df_clusters.groupby("cluster")[features_used].mean(numeric_only=
    True).add_suffix("_mean")
profile_med = df_clusters.groupby("cluster")[features_used].median(numeric_only=
    True).add_suffix("_median")

profile = pd.concat([profile_count, profile_mean, profile_med], axis=1).sort_index
    ()
display(profile.round(2))

# save profile and labeled dataset
profile.to_csv(out_dir / "06_cluster_profile.csv")
(df_clusters
    .to_csv(out_dir / "products_with_clusters.csv", index=False))

print("Saved:")
print(" -", out_dir / "04_pca_2d.png")
print(" -", out_dir / "05_silhouette_bars.png")
print(" -", out_dir / "centers_logspace.csv")
print(" -", out_dir / "centers_original_units.csv")
print(" -", out_dir / "06_cluster_profile.csv")
print(" -", out_dir / "products_with_clusters.csv")

from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # noqa: F401 (for 3D projection)

pca3 = PCA(n_components=3, random_state=42)
proj3 = pca3.fit_transform(X_scaled)

fig = plt.figure(figsize=(8,7))
ax = fig.add_subplot(111, projection="3d")

for c in range(k_best):
    idx = labels == c

```



```

        ax.scatter(
            proj3[idx,0], proj3[idx,1], proj3[idx,2],
            s=30, alpha=.85, label=f"Cluster {c}"
        )

ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_zlabel("PC3")
ax.set_title("3D PCA Projection      K-Means Clusters")
ax.legend()
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/3d_pca_projection_k-means.png",
            dpi=300, bbox_inches="tight")
plt.show()

print("Explained variance ratio (3 components):", pca3.explained_variance_ratio_)

!pip -q install fastcluster

## Hierarchical Clustering

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from pathlib import Path

from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score, silhouette_samples
from sklearn.decomposition import PCA

from scipy.cluster.hierarchy import linkage, dendrogram, fcluster

# Optional (faster linkage on big data if installed)
try:
    import fastcluster
except:
    fastcluster = None

np.random.seed(42)

base = "/content/drive/MyDrive/Dat Capstone"
out_dir = Path(base) / "hierarchical_outputs"
out_dir.mkdir(parents=True, exist_ok=True)

#Load data

df = pd.read_csv(f"{base}/cleaned_makeup_products.csv", low_memory=False)
print("Shape:", df.shape)
df.head(3)

#4) Impute, transform, and build a preprocessing pipeline

```

```

# numeric = number dtypes
num_cols = df.select_dtypes(include=["number"]).columns.tolist()
# categorical = object/category/bool (cast bool to category for OHE)
cat_cols = df.select_dtypes(include=["object", "category", "bool"]).columns.tolist()
()

# Work on a copy to keep originals
work = df.copy()

# Simple median impute for numeric
work[num_cols] = work[num_cols].fillna(work[num_cols].median(numeric_only=True))

# Build a ColumnTransformer
ohe = OneHotEncoder(sparse_output=False, handle_unknown="ignore", drop="first")
pre = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_cols),
        ("cat", ohe, cat_cols)
    ],
    remainder="drop"
)

# Fit/transform to get final feature matrix
X = pre.fit_transform(work)

# Get feature names for later (optional, nice for profiling)
num_names = num_cols
cat_names = []
if len(cat_cols):
    cat_names = list(pre.named_transformers_["cat"].get_feature_names_out(cat_cols))
feature_names = num_names + cat_names

print("Final feature matrix shape:", X.shape)

import numpy as np

print("X shape:", X.shape)
print("Any NaN in X? ", np.isnan(X).any())
print("Any non-finite X? ", ~np.isfinite(X).all())
# Which rows/cols are bad (optional):
bad_rows = ~np.isfinite(X).all(axis=1)
bad_cols = ~np.isfinite(X).all(axis=0)
print("Bad rows:", bad_rows.sum(), "Bad cols:", bad_cols.sum())

#4b) Rebuild preprocessing robustly (impute      drop zero-variance      scale
      one-hot)

#Rebuild preprocessing robustly (impute      drop zero-variance      scale      one-
      hot)

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.feature_selection import VarianceThreshold

```

```

import numpy as np
import pandas as pd

# Start from the original df, and num_cols / cat_cols you already defined earlier
work = df.copy()

# Ensure numeric dtype
for c in num_cols:
    work[c] = pd.to_numeric(work[c], errors="coerce")

# Pipelines
num_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("var", VarianceThreshold(threshold=0.0)), # drop zero-variance numerics (
        prevents scaler NaNs)
    ("scaler", StandardScaler())
])

cat_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")), # fill missing
        categories
    ("ohe", OneHotEncoder(handle_unknown="ignore", drop="first", sparse_output=
        False))
])

pre = ColumnTransformer(
    transformers=[
        ("num", num_pipe, num_cols),
        ("cat", cat_pipe, cat_cols)
    ],
    remainder="drop"
)

X = pre.fit_transform(work)

# ----- Recover feature names (after VT + OHE) -----
# Numeric names after variance threshold
num_mask = pre.named_transformers_["num"].named_steps["var"].get_support()
num_names_kept = [col for col, keep in zip(num_cols, num_mask) if keep]

# Categorical names from OHE
cat_names = []
if len(cat_cols):
    cat_names = list(pre.named_transformers_["cat"].named_steps["ohe"].
        get_feature_names_out(cat_cols))

feature_names = num_names_kept + cat_names

# Final safety: coerce any stray non-finites (should be none after VT)
if not np.isfinite(X).all():
    print("Warning: non-finite values found post-transform. Coercing with
        nan_to_num.")
    X = np.nan_to_num(X, nan=0.0, posinf=0.0, neginf=0.0)

print("Final feature matrix shape:", X.shape)
print("All finite?", np.isfinite(X).all())

```

```

print("Numeric features kept:", len(num_names_kept), " | Categorical features:",
      len(cat_names))

#5) Linkage matrix (Z) and last merges plot

from scipy.cluster.hierarchy import linkage, dendrogram
import matplotlib.pyplot as plt
from pathlib import Path

# Ward linkage on dense, finite matrix X
Z = linkage(X, method="ward")

# Inspect last 50 merge distances
plt.figure(figsize=(6,4))
last = Z[-50:, 2]
plt.plot(range(1, 51), last, marker="o")
plt.title("Last 50 Merge Distances      Ward Linkage")
plt.xlabel("Merge step from the end")
plt.ylabel("Distance")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/
            last_50_merge_distances_ward_linkage.png",
            dpi=300, bbox_inches="tight")
plt.show()

#6) Truncated dendrogram (readable)

plt.figure(figsize=(10,6))
dendrogram(
    Z,
    truncate_mode="lastp",
    p=20,
    leaf_rotation=90.,
    leaf_font_size=10.,
    show_contracted=True
)
plt.title("Hierarchical Clustering Dendrogram (truncated)")
plt.xlabel("Leaf cluster")
plt.ylabel("Distance")
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/
            hierarchical_clustering_dendrogram.png",
            dpi=300, bbox_inches="tight")
plt.show()

#7) Silhouette vs. k (2 12 ) with Agglomerative (Ward)

from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score

ks = range(2, 13)
sil_scores = []
for k in ks:
    labels_k = AgglomerativeClustering(n_clusters=k, linkage="ward").fit_predict(X

```

```

    )
    sil_scores.append(silhouette_score(X, labels_k))

plt.figure(figsize=(6,4))
plt.plot(list(ks), sil_scores, marker="o")
plt.title("Silhouette vs k      Hierarchical (Ward)")
plt.xlabel("k")
plt.ylabel("Silhouette score")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/silhouetter_vs_k_hierarchical.png",
            dpi=300, bbox_inches="tight")
plt.show()

for k, s in zip(ks, sil_scores):
    print(f"k={k:2d}    silhouette={s:.3f}")

#Pick k_best, fit final model, sizes, PCA 2D

from sklearn.decomposition import PCA

k_best = 4 # <-- adjust based on plots above

agg_final = AgglomerativeClustering(n_clusters=k_best, linkage="ward")
labels = agg_final.fit_predict(X)

work["cluster"] = labels
print("Cluster sizes:\n", work["cluster"].value_counts().sort_index())

# PCA projection
pca2 = PCA(n_components=2, random_state=42)
proj2 = pca2.fit_transform(X)

plt.figure(figsize=(7,6))
for c in range(k_best):
    idx = labels == c
    plt.scatter(proj2[idx,0], proj2[idx,1], s=25, alpha=0.85, label=f"Cluster {c}")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title(f"Hierarchical (Ward)      PCA 2D Projection (k={k_best})")
plt.legend()
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/hierarchical_pca_2d_projection.png",
            dpi=300, bbox_inches="tight")
plt.show()

print("Explained variance ratio (PC1, PC2):", pca2.explained_variance_ratio_)

#9) Silhouette bar plot per cluster

from sklearn.metrics import silhouette_samples
import numpy as np

```

```

sample_sil = silhouette_samples(X, labels)
y_lower = 10
plt.figure(figsize=(7,6))
for i in range(k_best):
    ith = sample_sil[labels == i]
    ith.sort()
    size_i = ith.shape[0]
    y_upper = y_lower + size_i
    plt.fill_betweenx(np.arange(y_lower, y_upper), 0, ith, alpha=0.7)
    plt.text(-0.05, y_lower + 0.5*size_i, str(i))
    y_lower = y_upper + 10

plt.axvline(np.mean(sample_sil), linestyle="--")
plt.xlabel("Silhouette coefficient")
plt.ylabel("Cluster")
plt.title(f"Silhouette Plot      Hierarchical (k={k_best})")
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/silhouetter_plot_hierarchical.png",
            dpi=300, bbox_inches="tight")
plt.show()

# Numeric summaries
profile_count = work["cluster"].value_counts().sort_index().rename("count")
profile_mean  = work.groupby("cluster")[num_names_kept].mean(numeric_only=True).
    add_suffix("_mean")
profile_med   = work.groupby("cluster")[num_names_kept].median(numeric_only=True).
    add_suffix("_median")
profile = pd.concat([profile_count, profile_mean, profile_med], axis=1)

# Optional: top 3 categories per cluster
cat_top = {}
for c in cat_cols:
    cat_top[c] = work.groupby("cluster")[c].agg(lambda s: s.value_counts().head(3))

# Save
profile.to_csv(out_dir/"06_cluster_profile_hierarchical.csv")
labeled = df.copy()
labeled["cluster"] = labels
labeled.to_csv(out_dir/"products_with_clusters_hierarchical.csv", index=False)

print("Saved:")
print(" -", out_dir/"01_last_merges.png")
print(" -", out_dir/"02_dendrogram_truncated.png")
print(" -", out_dir/"03_silhouette_vs_k_hierarchical.png")
print(" -", out_dir/"04_pca2_hierarchical.png")
print(" -", out_dir/"05_silhouetteBars_hierarchical.png")
print(" -", out_dir/"06_cluster_profile_hierarchical.csv")
print(" -", out_dir/"products_with_clusters_hierarchical.csv")

from scipy.cluster.hierarchy import linkage, dendrogram

subset_n = min(400, X.shape[0])
idx = np.random.choice(X.shape[0], subset_n, replace=False)
Z_sub = linkage(X[idx], method="ward")

```

```

plt.figure(figsize=(12,6))
dendrogram(Z_sub, leaf_rotation=90., leaf_font_size=7.)
plt.title(f"Full Dendrogram (subset n={subset_n})")
plt.xlabel("Samples")
plt.ylabel("Distance")
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/full_dendrogram.png",
            dpi=300, bbox_inches="tight")
plt.show()

!pip -q install pyclustering

import pandas as pd
base = "/content/drive/MyDrive/Dat Capstone" # same path you used earlier
csv_path = f"{base}/cleaned_makeup_products.csv"

# Load your products dataset
df = pd.read_csv(csv_path, low_memory=False)

# Drop text / ID / non-numeric columns
drop_cols = [
    "product_link_id", "product_link", "item_id", "product_name",
    "description", "pros", "cons", "best_uses", "describe_yourself", "brand", "category"
]
drop_cols = [c for c in drop_cols if c in df.columns]

# Keep only numeric features
num_cols = [c for c in df.columns if c not in drop_cols]
for c in num_cols:
    df[c] = pd.to_numeric(df[c], errors="coerce")

numeric_df = df.loc[:, num_cols].dropna(axis=1, how="all")
numeric_df.head()

X = numeric_df.fillna(numeric_df.median(numeric_only=True)).copy()
...

import numpy as np
from sklearn.preprocessing import StandardScaler

# Fill missing values with medians
X = numeric_df.fillna(numeric_df.median(numeric_only=True)).copy()

# Log-transform long-tailed columns (the large-count ones)
skew_cols = [c for c in X.columns if (X[c].max() > 100) and (X[c].min() >= 0)]
for c in skew_cols:
    X[c] = np.log1p(X[c])

# Standardize all numeric features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("X_scaled shape:", X_scaled.shape)

from pyclustering.cluster.kmedoids import kmedoids

```

```

from pyclustering.utils.metric import distance_metric, type_metric
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
metric = distance_metric(type_metric.EUCLIDEAN)

def run_kmedoids_for_k(X_scaled, k, n_init=10):
    best = {"sil": -1, "labels": None, "medoids": None}
    n = X_scaled.shape[0]
    for _ in range(n_init):
        init = rng.choice(n, size=k, replace=False).tolist()
        algo = kmedoids(X_scaled.tolist(), init, metric=metric)
        algo.process()
        clusters = algo.get_clusters()
        labels = np.full(n, -1, dtype=int)
        for cid, idxs in enumerate(clusters):
            labels[idxs] = cid
        if (labels >= 0).all():
            sil = silhouette_score(X_scaled, labels)
            if sil > best["sil"]:
                best = {"sil": sil, "labels": labels, "medoids": algo.get_medoids()}
    return best

def total_cost(X_scaled, labels, medoids):
    M = X_scaled[medoids]
    d = np.sqrt(((X_scaled[:,None,:] - M[None,:,:])**2).sum(axis=2)).min(axis=1)
    return d.sum()

ks = range(2, 13)
costs, sils, best_runs = [], [], []
for k in ks:
    res = run_kmedoids_for_k(X_scaled, k, n_init=10)
    best_runs.append(res)
    costs.append(total_cost(X_scaled, res["labels"], res["medoids"]))
    sils.append(res["sil"])

plt.figure(figsize=(6,4))
plt.plot(list(ks), costs, marker="o")
plt.xlabel("k"); plt.ylabel("Sum distance to medoid")
plt.title("Elbow K-Medoids (PAM)")
plt.grid(alpha=.3);
plt.savefig("/content/drive/MyDrive/Dat Capstone/kmedoids_elbow.png",
            dpi=300, bbox_inches="tight")
plt.show()

plt.figure(figsize=(6,4))
plt.plot(list(ks), sils, marker="o", color="orange")
plt.xlabel("k"); plt.ylabel("Silhouette Score")
plt.title("Silhouette vs k K-Medoids (PAM)")
plt.grid(alpha=.3);
plt.savefig("/content/drive/MyDrive/Dat Capstone/kmedoids_silhouette_vs_k.png",
            dpi=300, bbox_inches="tight")
plt.show()

```



```

for k, s in zip(ks, sils):
    print(f"k={k}, silhouette={s:.3f}")

from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_samples
import numpy as np

k_best = 4 # change this if a different k gives best silhouette
best = best_runs[k_best - ks.start]
labels, medoids = best["labels"], best["medoids"]

df_kmed = df.copy()
df_kmed["cluster"] = labels

print("Cluster sizes:\n", df_kmed["cluster"].value_counts().sort_index())

# Cluster summary
summary = df_kmed.groupby("cluster")[X.columns].mean().round(2)
display(summary)

# PCA plot
pca = PCA(n_components=2, random_state=42)
proj2 = pca.fit_transform(X_scaled)
plt.figure(figsize=(7,6))
for c in range(k_best):
    plt.scatter(proj2[labels==c,0], proj2[labels==c,1], s=30, alpha=.85, label=f"
                Cluster {c}")
plt.xlabel("PC1"); plt.ylabel("PC2")
plt.title(f"K-Medoids (k={k_best}) PCA 2D Projection")
plt.legend();
plt.savefig("/content/drive/MyDrive/Dat Capstone/kmedoids_pca2.png",
            dpi=300, bbox_inches="tight")
plt.show()

#Show the actual medoid products (representatives).

df.iloc[medoids][["product_name","brand","price","average_rating"]]

#Profile clusters (which features separate them most).
Quick stats + effect sizes:
import pandas as pd, numpy as np
prof_mean = df_kmed.groupby("cluster")[X.columns].mean().round(2)
prof_med = df_kmed.groupby("cluster")[X.columns].median().round(2)
prof_iqr = df_kmed.groupby("cluster")[X.columns].quantile([.25,.75]).unstack(0)
display(prof_mean); display(prof_med); display(prof_iqr)

"""--- Extended EDA ---"""

import seaborn as sns
import matplotlib.pyplot as plt
sns.set(style="whitegrid", palette="muted", font_scale=1.1)
plt.rcParams['figure.figsize'] = (8, 6)

plt.figure(figsize=(10,6))
sns.boxplot(data=products, x="category", y="rating", palette="coolwarm")
plt.xticks(rotation=45, ha='right')

```

```

plt.title("Price vs Rating by Product Category")
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/price_vs_rating_by_category.png",
            dpi=300, bbox_inches="tight")
plt.show()

# Clean + prepare
reviews['created_date'] = pd.to_datetime(reviews['created_date'], errors='coerce')
reviews['updated_date'] = pd.to_datetime(reviews['updated_date'], errors='coerce')
reviews['year'] = reviews['created_date'].dt.year
reviews['review_length'] = reviews['comments'].astype(str).str.len()

# --- FIX DATETIME ---
for c in ['created_date', 'updated_date']:
    reviews[c] = pd.to_datetime(reviews[c], errors='coerce', infer_datetime_format
                                =True)

# --- CREATE YEAR COLUMN ---
reviews['year'] = reviews['created_date'].dt.year

trend = reviews.groupby('year').size().reset_index(name='review_count')
sns.lineplot(data=trend, x='year', y='review_count', marker='o')
plt.title('Review Volume Over Time')
plt.xlabel('Year'); plt.ylabel('Number of Reviews')
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/review_volume_over_time.png",
            dpi=300, bbox_inches="tight")
plt.show()

sns.scatterplot(data=reviews, x='rating', y='helpful_votes', alpha=0.6)
plt.title('Helpful Votes vs Rating')
plt.xlabel('Rating'); plt.ylabel('Helpful Votes')
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/helpful_votes_vs_rating.png",
            dpi=300, bbox_inches="tight")
plt.show()

num_cols = ['rating', 'helpful_votes', 'not_helpful_votes', 'helpful_score',
            'review_length']
corr = reviews[num_cols].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f', square=True)
plt.title('Correlation Heatmap of Review Metrics')
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Dat Capstone/
            correlation_heatmap_of_review_metrics.png",
            dpi=300, bbox_inches="tight")
plt.show()

positive_text = " ".join(reviews.loc[reviews['rating']>=4, 'comments'].dropna().
                        astype(str))
negative_text = " ".join(reviews.loc[reviews['rating']<=2, 'comments'].dropna().
                        astype(str))

wc_pos = WordCloud(width=800, height=400, background_color='white', colormap='
                    Blues').generate(positive_text)
wc_neg = WordCloud(width=800, height=400, background_color='white', colormap='Reds

```

```

        ').generate(negative_text)

fig, ax = plt.subplots(1, 2, figsize=(14,6))
ax[0].imshow(wc_pos); ax[0].set_title('Positive Reviews'); ax[0].axis('off')
ax[1].imshow(wc_neg); ax[1].set_title('Negative Reviews'); ax[1].axis('off')
plt.savefig("/content/drive/MyDrive/Dat Capstone/wordCloud.png",
            dpi=300, bbox_inches="tight")
plt.show()

"""--Sentiment Analysis--"""

!pip install transformers datasets torch seaborn wordcloud scikit-learn --quiet

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import nltk

from sklearn.metrics import classification_report, accuracy_score
from datasets import Dataset
from transformers import AutoTokenizer, AutoModelForSequenceClassification,
    Trainer, TrainingArguments, pipeline
import evaluate
import torch
from tqdm import tqdm
from wordcloud import WordCloud
from nltk.corpus import stopwords
from collections import Counter

nltk.download('stopwords')
stop_words = set(stopwords.words('english'))

redundant_words = set([
    "product", "makeup", "foundation", "concealer", "lipstick",
    "mascara", "powder", "shade", "color", "brand", "buy", "bought",
    "thing", "stuff", "item", "review", "use", "used", "using",
    "work", "works", "like"
])

def clean_text(text):
    text = str(text).lower()

    for brand in brand_list:
        if len(brand.split()) > 8:
            text = text.replace(brand, "")

    # Remove punctuation and digits
    text = re.sub(r'[^a-zA-Z ]', ' ', text)

    # Tokenize
    tokens = text.split()

    # Remove stopwords, redundant words, and short words
    tokens = [

```

```

        w for w in tokens
        if w not in stop_words
        and w not in redundant_words
        and len(w) > 2
    ]
    return tokens

from transformers import pipeline



## B.2 BERT-based Sentiment Analysis and Extended Results



sentiment_model = pipeline("sentiment-analysis", model="nlptown/bert-base-multilingual-uncased-sentiment")

def get_sentiment(texts):
    results = []
    for text in tqdm(texts):
        if isinstance(text, str) and text.strip():
            result = sentiment_model(text[:512])[0] # truncate long text
            results.append(result['label'])
        else:
            results.append(None)
    return results

df['positive_sentiment'] = get_sentiment(df['faceoff_positive'])
df['negative_sentiment'] = get_sentiment(df['faceoff_negative'])

# Convert to numeric score from range of 1-5
df['faceoff_positive_score'] = df['positive_sentiment'].str.extract('(\d)').astype(float)
df['faceoff_negative_score'] = df['negative_sentiment'].str.extract('(\d)').astype(float)

pos_df = df[['faceoff_positive_score']].copy()
pos_df['text'] = df['faceoff_positive']
pos_df['true_label'] = 'positive'
pos_df.rename(columns={'faceoff_positive_score': 'score'}, inplace=True)

neg_df = df[['faceoff_negative_score']].copy()
neg_df['text'] = df['faceoff_negative']
neg_df['true_label'] = 'negative'
neg_df.rename(columns={'faceoff_negative_score': 'score'}, inplace=True)

combined_df = pd.concat([pos_df, neg_df]).dropna()

combined_df['predicted_label'] = combined_df['score'].apply(lambda x: 'positive'
    if x >= 3 else 'negative')

accuracy = accuracy_score(combined_df['true_label'], combined_df['predicted_label'])
print(f"BERT sentiment model accuracy: {accuracy:.2%}")

report = classification_report(
    combined_df['true_label'],
    combined_df['predicted_label'],
    target_names=['negative', 'positive']

```

```

)
print("Classification Report:\n", report)

# Sentiment Distribution

plt.figure(figsize=(8,5))
sns.countplot(x='score', hue='true_label', data=combined_df)
plt.title("Sentiment Rating Distribution (1-5)")
plt.xlabel("Predicted Score")
plt.ylabel("Count")
plt.legend(title="Label")
plt.show()

# Average sentiment by review type

plt.figure(figsize=(6,4))
sns.barplot(data=combined_df, x='true_label', y='score', palette='coolwarm', ci=
    None)
plt.title("Average Sentiment Score per Review Type")
plt.xlabel("Known Review Polarity")
plt.ylabel("Average BERT Sentiment Score")
plt.show()

# Confusion matrix (true vs predicted)

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(combined_df['true_label'], combined_df['predicted_label'],
    labels=['positive', 'negative'])
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Positive', '
    Negative'])
disp.plot(cmap='Blues')
plt.title("BERT Sentiment Confusion Matrix")
plt.show()

# Example of using model to predict future new makeup reviews

new_reviews = [
    "This lipstick is amazing! Very pigmented and long-lasting.",
    "The foundation is patchy and dries my skin.",
    "Neutral product, nothing special."
]

for review in new_reviews:
    result = sentiment_model(review)[0]
    print(f"Review: {review}")
    print(f"Predicted sentiment: {result['label']}, Confidence: {result['score
        ']:.2f}\n")

# Top Brands by Average Sentiment Plot

if 'brand' in df.columns and 'faceoff_positive_score' in df.columns:

    df['sentiment_score'] = df[['faceoff_positive_score', 'faceoff_negative_score'
        ]].mean(axis=1)

```

```

# Clean brand names
clean_brands = df['brand'].astype(str).str.lower()
clean_brands = (
    clean_brands
    .str.replace(r'creamy texture', '', regex=True)
    .str.replace(r'smooth blend', '', regex=True)
    .str.replace(r'smooth consistency', '', regex=True)
    .str.replace(r'smooth coverage', '', regex=True)
    .str.replace(r'smooth finish', '', regex=True)
    .str.replace(r'smooth consistency', '', regex=True)
    .str.strip()
)
df['brand'] = clean_brands

# Drop empty brand names
df_brand_plot = df[df['brand'] != '']

# Group by brand and get average sentiment
brand_sentiment = (
    df_brand_plot.groupby('brand')['sentiment_score']
    .mean()
    .sort_values(ascending=False)
    .head(10)
    .reset_index()
)

plt.figure(figsize=(10,5))
sns.barplot(x='sentiment_score', y='brand', data=brand_sentiment, palette='
Greens_r')
plt.title("Top 10 Brands by Average Sentiment")
plt.xlabel("Average Sentiment (1 5 Stars)")
plt.ylabel("Brand")
plt.show()

# Worst Brands by Average Sentiment Plot

worst_brands = (
    df_brand_plot.groupby('brand')['sentiment_score']
    .mean()
    .sort_values(ascending=True)
    .head(10)
    .reset_index()
)

plt.figure(figsize=(10,5))
sns.barplot(
    x='sentiment_score', y='brand',
    data=worst_brands, palette='Reds_r'
)
plt.title("Bottom 10 Brands by Average Sentiment")
plt.xlabel("Average Sentiment (1 5 Stars)")
plt.ylabel("Brand")
plt.show()

# Top Products Based on Review Sentiment

```

```

if 'product_name' in df.columns:
    clean_products = df['product_name'].astype(str).str.lower()

    phrases_to_remove = [
        r'long[-\s]?lasting',
        r'similar to tarte shape tape \((which i love\)',
        r'similar to tarte shape tape',
        r'sits light on your face',
        r'size',
        r'skin match',
        r'skin tone match',
        r'slight matte finish',
        r'small amount goes a long way',
        r'small amount goes long way',
        r'small amounts?',
        r'small container',
        r'smell',
        r'smells great',
        r'smooth and very lightweight!?',
        r'smooth and creaseless',
        r'smooth and long lasting',
        r'smooth and velvet[-\s]?y feel',
        r'smooth application',
        r'smooth finish',
        r'smooth texture',
        r'smooth\b',
        r'small\b',
    ]

    for phrase in phrases_to_remove:
        clean_products = clean_products.str.replace(phrase, '', regex=True)

    clean_products = clean_products.str.replace(r'\s+', ' ', regex=True).str.strip()
    clean_products = clean_products.replace('', np.nan).fillna(method='bfill')

    df_product_plot = df.copy()
    df_product_plot['product_name'] = clean_products

    product_sent = (
        df_product_plot.groupby('product_name')['sentiment_score']
        .mean()
        .sort_values(ascending=False)
        .head(10)
        .reset_index()
    )

    # Plot
    plt.figure(figsize=(10, 5))
    sns.barplot(x='sentiment_score', y='product_name', data=product_sent, palette='coolwarm')
    plt.title("Top 10 Products by Average Sentiment")
    plt.xlabel("Average Sentiment (1 5 Stars)")
    plt.ylabel("Product")
    plt.show()

```